Step 5 — The Full Team: a Supervisor Routing 4 Specialists via Command — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 5: four specialists, each already proven alone in Steps 1-4. Now a **Supervisor** node runs the whole team: it reads the shared state, decides which agent works next, logs *why* it chose that agent, and routes directly with LangGraph's `Command(goto=...)` (Doc11's pattern), instead of fixed graph edges. The Writer ↔ Reviewer loop moves from Step 2's `main.py` `while` loop into the graph itself — same `MAX_REVISIONS` limit, same `PASS_SCORE`. Crucially, the agents' inner `run_...()` functions don't change at all; each just gets a small node wrapper around the function you already tested. **If not:** the order of work would stay hard-coded in `main.py`, with no shared state to inspect and no routing log to explain a bad run.

After this step the project is complete. Files not shown here don't change.

Solution — a testable routing function, a reason for every move, one writer per field

Story — `state.py`: this is the shared `TypedDict` state, built with **one writer per field** (Doc11's rule) — Research is the only node that writes the notes field, Analysis the only one that writes findings, and so on; only the Supervisor writes the routing log. `research_done` is a separate boolean that records Research *ran* even when it found nothing, which is what stops the Supervisor from sending the same topic back to Research forever. **If not:** one agent's working notes could overwrite another's, and the Supervisor couldn't tell "not run yet" from "ran, found nothing."

```
# src/state.py
# TeamState: the shared state every node reads and writes, with ONE writer
# per field (Doc11). start_state(topic) gives every field a start value.
from typing import TypedDict

class TeamState(TypedDict):
    topic: str                  # shared input — every agent may read it
    research_notes: str         # written by Research
    research_done: bool         # written by Research
    findings: str               # written by Analysis
    draft: str                  # written by Writer (cleared by Reviewer)
    feedback_history: list[str]  # written by Reviewer
```

```
    accepted: bool               # written by Reviewer
    revision_count: int          # written by Reviewer
    routing_log: list[dict]      # written by the Supervisor only

def start_state(topic):
    return {
        "topic": topic,
        "research_notes": "",
        "research_done": False,
        "findings": "",
        "draft": "",
        "feedback_history": [],
        "accepted": False,
        "revision_count": 0,
        "routing_log": [],
    }
```

Story — `agents/supervisor.py`: `decide_next()` is a plain function of the state — no model call — that returns `(next step, reason)`, which makes it testable directly with a hand-made state dict, no graph needed. `supervisor()` wraps it as a node: logs the decision, then returns `Command(update=..., goto=...)`, which both updates the state and picks the next node in one return value. The *order* the rules are checked in **is** the whole routing policy: research first → stop if nothing was found → analysis → stop if the draft is accepted or the revision limit is hit → writer → reviewer. **If not:** routing bugs could only be found by running the whole team end to end, every time.

```
# src/agents/supervisor.py
# The Supervisor. MAX_REVISIONS caps the Writer<->Reviewer loop.
# decide_next(state) -> (step, reason) is the routing policy, in order.
# supervisor(state) logs the decision and routes with Command.
from langgraph.graph import END
from langgraph.types import Command

from src.utils.logger import get_logger

MAX_REVISIONS = 3

def decide_next(state):
    if not state["research_done"]:
        return "research_agent", "no research yet"
    if state["research_notes"] == "":
        return "END", "research found no facts for this topic"
    if state["findings"] == "":
        return "analysis_agent", "research done, no findings yet"
    if state["accepted"]:
        return "END", "reviewer accepted the draft"
    if state["revision_count"] >= MAX_REVISIONS:
```

```
        return "END", f"revision limit ({MAX_REVISIONS}) reached"
    if state["draft"] == "":
        if state["feedback_history"]:
            return "writer_agent", "revising after reviewer feedback"
        return "writer_agent", "findings ready, no draft yet"
    return "reviewer_agent", "draft ready for review"

def supervisor(state):
    logger = get_logger(__name__)
    step, reason = decide_next(state)
    logger.info(f"supervisor -> {step} ({reason})")
    log = state["routing_log"] + [{"step": step, "reason": reason}]
    if step == "END":
        return Command(update={"routing_log": log}, goto=END)
    return Command(update={"routing_log": log}, goto=step)
```

Changed from Steps 1-4: each agent file gets a small **node wrapper** that reads its inputs from the state, calls the unchanged `run_...()` function, and returns `Command(update=..., goto="supervisor")`. The `run_...()` functions themselves do not change.

```
# src/agents/research_agent.py
# The Research agent. run_research(topic) looks facts up first; research_node
# wraps it for the graph and marks research_done so it never runs twice.
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langgraph.types import Command

from src.models.llm import get_model
from src.tools.lookup import lookup_facts
from src.utils.logger import get_logger
from src.utils.prompts import load_prompt

def run_research(topic):
    logger = get_logger(__name__)
    facts = lookup_facts.invoke({"topic": topic})
    if facts == "":
        logger.warning(f"no facts found for {topic!r}")
        return ""
    prompt = ChatPromptTemplate.from_template(load_prompt("research"))
    chain = prompt | get_model() | StrOutputParser()
    return chain.invoke({"topic": topic, "facts": facts})

# new in Step 5: the node wrapper
def research_node(state):
    notes = run_research(state["topic"])
```

```
    return Command(update={"research_notes": notes, "research_done": True},
                   goto="supervisor")
```

```
# src/agents/analysis_agent.py
# The Analysis agent. run_analysis(notes) groups the notes into themes;
# analysis_node wraps it for the graph.
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langgraph.types import Command

from src.models.llm import get_model
from src.utils.prompts import load_prompt

def run_analysis(notes):
    prompt = ChatPromptTemplate.from_template(load_prompt("analysis"))
    chain = prompt | get_model() | StrOutputParser()
    return chain.invoke({"notes": notes})

# new in Step 5: the node wrapper
def analysis_node(state):
    findings = run_analysis(state["research_notes"])
    return Command(update={"findings": findings}, goto="supervisor")
```

```
# src/agents/writer_agent.py
# The Writer agent. run_writer(findings, feedback) drafts the article;
# writer_node passes the newest Reviewer feedback back in on a revision.
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langgraph.types import Command

from src.models.llm import get_model
from src.utils.prompts import load_prompt

def build_writer_chain():
    prompt = ChatPromptTemplate.from_template(load_prompt("writer"))
    return prompt | get_model() | StrOutputParser()

def run_writer(findings, feedback=""):
    if feedback == "":
        feedback = "none"
    chain = build_writer_chain()
    return chain.invoke({"findings": findings, "feedback": feedback})

# new in Step 5: the node wrapper
def writer_node(state):
```

```
    feedback = ""
    if state["feedback_history"]:
        feedback = state["feedback_history"][-1]
    draft = run_writer(state["findings"], feedback)
    return Command(update={"draft": draft}, goto="supervisor")
```

Story — `reviewer_node`: this is Step 2's loop logic, now as a graph node instead of a `while` loop body. On a pass it sets `accepted` in the state. On a fail it adds the feedback to the history and counts the revision — and crucially, while revisions are still left, it **clears the draft** field. An empty draft is exactly what the Supervisor's routing rules read as "needs the Writer again" — that's the mechanism that sends work back (Doc11's pattern). `MAX_REVISIONS` now lives in the Supervisor, the one place that decides when to stop the loop entirely. **If not:** the loop would have no memory of past feedback, or no way back to the Writer.

```
# src/agents/reviewer_agent.py
# The Reviewer agent. PASS_SCORE is the pass mark; run_reviewer returns a
# checked ReviewVerdict; reviewer_node records the verdict and, while
# revisions remain, clears the draft to send it back to the Writer.
from langchain_core.prompts import ChatPromptTemplate
from langgraph.types import Command

from src.agents.supervisor import MAX_REVISIONS
from src.models.llm import get_model
from src.schemas import ReviewVerdict
from src.utils.logger import get_logger
from src.utils.prompts import load_prompt

PASS_SCORE = 7

def run_reviewer(findings, draft):
    prompt = ChatPromptTemplate.from_template(load_prompt("reviewer"))
    chain = prompt | get_model().with_structured_output(ReviewVerdict)
    return chain.invoke({"findings": findings, "draft": draft})

# new in Step 5: Step 2's loop, as a node
def reviewer_node(state):
    logger = get_logger(__name__)
    verdict = run_reviewer(state["findings"], state["draft"])
    logger.info(f"review score {verdict.score}: {verdict.feedback}")

    if verdict.score >= PASS_SCORE:
        return Command(update={"accepted": True}, goto="supervisor")

    new_count = state["revision_count"] + 1
    history = state["feedback_history"] + [verdict.feedback]
    update = {"revision_count": new_count, "feedback_history": history}
```

```
    if new_count < MAX_REVISIONS:
        update["draft"] = ""
    return Command(update=update, goto="supervisor")
```

Story — `graph.py`: this graph has five nodes but only **one** fixed edge (`START → supervisor`). Every other move between nodes is a `Command(goto=...)` returned by a node at run time, not a pre-drawn edge — that's what makes the Supervisor an actual decision-maker rather than a fixed pipeline wearing a graph's clothes. **If not:** you'd draw fixed edges for every transition, and the Supervisor would have nothing left to decide.

```
# src/graph.py
# build_graph(): five nodes and one edge (START -> supervisor). Every other
# move is a Command(goto=...) a node returns, so the Supervisor routes at
# run time.
from langgraph.graph import START, StateGraph

from src.agents.analysis_agent import analysis_node
from src.agents.research_agent import research_node
from src.agents.reviewer_agent import reviewer_node
from src.agents.supervisor import supervisor
from src.agents.writer_agent import writer_node
from src.state import TeamState

def build_graph():
    builder = StateGraph(TeamState)
    builder.add_node("supervisor", supervisor)
    builder.add_node("research_agent", research_node)
    builder.add_node("analysis_agent", analysis_node)
    builder.add_node("writer_agent", writer_node)
    builder.add_node("reviewer_agent", reviewer_node)
    builder.add_edge(START, "supervisor")
    return builder.compile()
```

Changed from Step 4: `print_report()` keeps its name and input — it only **gains** two things: the routing log at the top, and a "nothing is known" branch (the team can now stop right after an empty research). `main()` runs the graph instead of calling the agents one by one. And `write_with_review()` is removed: its job — write, review, send back, stop at the limit — is now done by the Supervisor and `reviewer_node`. That is the one function in the project that gets replaced, because the whole point of this step is handing that loop to the team.

```
# main.py
# The file you run: python main.py
# Step 5 version: build the graph, read a topic, run the team, and print the
# routing log and the result.
import sys
```

```
import openai

from src.config import load_config
from src.exceptions import MissingConfigError
from src.graph import build_graph
from src.state import start_state
from src.utils.logger import get_logger

def print_report(result):
    # new in Step 5: the routing log
    print("Routing log:")
    for entry in result["routing_log"]:
        print(f"  {entry['step']}: {entry['reason']}")
    if result["accepted"]:
        print("\nFinal article:\n")
        print(result["draft"])
    # new in Step 5: the team can stop after an empty research
    elif result["research_notes"] == "":
        print("\nNo article: nothing is known about this topic yet.")
    else:
        attempts = result["revision_count"]
        print(f"\nCouldn't agree after {attempts} revisions. Feedback:")
        number = 1
        for feedback in result["feedback_history"]:
            print(f"  round {number}: {feedback}")
            number = number + 1
        print("\nLast draft (NOT approved):\n")
        print(result["draft"])

def main():
    try:
        load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    logger = get_logger(__name__)
    graph = build_graph()
    topic = input("Topic: ").strip()
    try:
        result = graph.invoke(start_state(topic))
    except openai.AuthenticationError:
        logger.error("Authentication failed — check OPENAI_API_KEY.")
        print("Login failed — check OPENAI_API_KEY in your .env file.")
        sys.exit(1)
    print_report(result)
```

```
if __name__ == "__main__":
    main()
```

The Step 5 tests

Story — `tests/unit/test_supervisor.py`: this checks `decide_next()`'s routing rules against hand-made states — every step in its expected order, and every way the run can stop (nothing found, accepted, limit hit) — with no model, no graph, and no API key needed, since routing is pure Python. **If not:** a wrong rule order would only show up as a strange, expensive run through the whole team.

```
# tests/unit/test_supervisor.py
# Checks the Supervisor's routing rules with hand-made states — no model,
# no graph, no API key. Each case prints OK or WRONG.
# Run from the repo root: python -m tests.unit.test_supervisor
from src.agents.supervisor import MAX_REVISIONS, decide_next
from src.state import start_state


def check(label, state, expected):
    step, reason = decide_next(state)
    result = "OK"
    if step != expected:
        result = "WRONG"
    print(f"{label} -> {result} ({step})")


def test_routing_order():
    state = start_state("electric bikes")
    check("case 1 (fresh start)", state, "research_agent")

    state["research_done"] = True
    state["research_notes"] = "notes"
    check("case 2 (research done)", state, "analysis_agent")

    state["findings"] = "findings"
    check("case 3 (findings ready)", state, "writer_agent")

    state["draft"] = "draft"
    check("case 4 (draft ready)", state, "reviewer_agent")


def test_stopping_rules():
    nothing_found = start_state("deep sea mining")
```

```
    nothing_found["research_done"] = True
    check("case 5 (nothing found)", nothing_found, "END")

    revising = start_state("electric bikes")
    revising["research_done"] = True
    revising["research_notes"] = "notes"
    revising["findings"] = "findings"
    revising["feedback_history"] = ["add sources"]
    revising["revision_count"] = 1
    check("case 6 (draft rejected)", revising, "writer_agent")

    revising["revision_count"] = MAX_REVISIONS
    revising["draft"] = "last draft"
    check("case 7 (limit reached)", revising, "END")

    revising["revision_count"] = 1
    revising["accepted"] = True
    check("case 8 (accepted)", revising, "END")


if __name__ == "__main__":
    test_routing_order()
    test_stopping_rules()
```

Changed from Step 4: the integration test keeps cases 1-5 and adds case 6 (a known topic goes through all four specialists) and case 7 (an unknown topic makes the team stop).

```
# tests/integration/test_live_agents.py
# Runs the agents and the whole team against the real OpenAI API — each step
# adds cases. Each case prints OK or WRONG. Needs OPENAI_API_KEY in .env.
# Run from the repo root: python -m tests.integration.test_live_agents
from src.agents.analysis_agent import run_analysis
from src.agents.research_agent import run_research
from src.agents.reviewer_agent import PASS_SCORE, run_reviewer
from src.agents.writer_agent import run_writer
from src.config import load_config
from src.graph import build_graph
from src.state import start_state

FINDINGS = (
    "Range:\n- A battery gives 40 to 100 km per charge. [battery-review]\n"
    "Rules:\n- EU e-bike motors are limited to 25 km/h. [eu-rules]\n"
)


def test_writer_keeps_sources():
```

```
    draft = run_writer(FINDINGS)
    result = "OK"
    if "[" not in draft:
        result = "WRONG"
    print(f"case 1 (writer keeps sources) -> {result}")


def test_reviewer_rejects_weak_draft():
    verdict = run_reviewer(FINDINGS, "E-bikes are good.")
    result = "OK"
    if verdict.score >= PASS_SCORE:
        result = "WRONG"
    print(f"case 2 (weak draft fails)     -> {result}")


def test_research_keeps_sources():
    notes = run_research("electric bikes")
    result = "OK"
    if notes.count("[") < 5:
        result = "WRONG"
    print(f"case 3 (known topic sourced)  -> {result}")


def test_research_unknown_topic():
    notes = run_research("deep sea mining")
    result = "OK"
    if notes != "":
        result = "WRONG"
    print(f"case 4 (unknown topic empty)  -> {result}")


def test_analysis_keeps_every_fact():
    notes = run_research("electric bikes")
    findings = run_analysis(notes)
    result = "OK"
    if findings.count("[") != notes.count("["):
        result = "WRONG"
    print(f"case 5 (analysis keeps facts) -> {result}")


def test_full_pipeline(graph):
    result_state = graph.invoke(start_state("electric bikes"))
    steps = []
    for entry in result_state["routing_log"]:
        steps.append(entry["step"])
    needed = ["research_agent", "analysis_agent", "writer_agent",
              "reviewer_agent"]
```

```
    result = "OK"
    for step in needed:
        if step not in steps:
            result = "WRONG"
    print(f"case 6 (full pipeline)        -> {result}")


def test_unknown_topic_stops(graph):
    result_state = graph.invoke(start_state("deep sea mining"))
    last = result_state["routing_log"][-1]
    result = "OK"
    if last["step"] != "END" or "no facts" not in last["reason"]:
        result = "WRONG"
    print(f"case 7 (unknown topic stops)  -> {result}")


if __name__ == "__main__":
    load_config()
    graph = build_graph()
    test_writer_keeps_sources()
    test_reviewer_rejects_weak_draft()
    test_research_keeps_sources()
    test_research_unknown_topic()
    test_analysis_keeps_every_fact()
    test_full_pipeline(graph)
    test_unknown_topic_stops(graph)
```

Run everything (from the repo root):

```
python -m tests.unit.test_config
python -m tests.unit.test_lookup
python -m tests.unit.test_supervisor
python -m tests.integration.test_live_agents
python main.py
```

Expected unit output (`test_supervisor`):

```
case 1 (fresh start) -> OK (research_agent)
case 2 (research done) -> OK (analysis_agent)
case 3 (findings ready) -> OK (writer_agent)
case 4 (draft ready) -> OK (reviewer_agent)
case 5 (nothing found) -> OK (END)
case 6 (draft rejected) -> OK (writer_agent)
case 7 (limit reached) -> OK (END)
case 8 (accepted) -> OK (END)
```

Expected app run (log lines left out; the article varies):

```
Topic: electric bikes
Routing log:
  research_agent: no research yet
  analysis_agent: research done, no findings yet
  writer_agent: findings ready, no draft yet
  reviewer_agent: draft ready for review
  writer_agent: revising after reviewer feedback
  reviewer_agent: draft ready for review
  END: reviewer accepted the draft

Final article:

E-Bikes: How Far, How Fast, and Why Europe Loves Them
...
```